Resuelve los siguientes ejercicios y entrega el código fuente java tanto en un fichero .pdf como en ficheros de texto, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. PROCESAR FICHEROS DE TEXTO CSV.

Un fichero .CSV es un fichero de texto plano con cero o más líneas. Cada línea representa un registro de información que describe algo (como una fila de una tabla de una BD relacional). Cada línea puede contener uno o más valores (o campos) separados por algún carácter delimitador (comas, o puntos y coma, o dos puntos según el caso). Los valores de texto pueden ir entrecomillados para evitar confusiones si el texto puede contener el carácter que se usa como separador de campos. A veces la primera línea del fichero se utiliza para etiquetar o describir los campos de las siguientes.

Para procesar este tipo de ficheros en Java, si el formato es sencillo: Leer el archivo de texto línea a línea, dividir cada línea en las cadenas de texto por el separador correspondiente usando el método `String.split(separador)` que devuelve un array de cadenas (mira documentación). Si conoces el formato de cada línea, puedes comprobar que tenga el nº de campos esperado, acceder y manipular cada campo y en definitiva, procesar el fichero.

Haz un programa llamado **U06E01** que pregunte por una provincia y muestre la información sobre sus espacios naturales. Para ello debe leer las líneas del fichero **espacios1.csv** que es un fichero de texto con la primera línea de comentario y el resto de líneas tiene 5 campos separados por el carácter ';' y que tiene este formato:

PROVINCIA;ESPACIO_NATURAL;CCAA;TIPO:SUPERFICIE

Además de la lógica del programa, puntúo que uses la clase **BufferedReader** (que tiene el método **readLine()**), que abras el fichero como solo lectura, que cada operación que pueda generar una excepción de entrada/salida se proteja mediante `catch(IOException e) { e.printStackTrace(); }` y que asegures que cierras el fichero al salir con una parte **finally{}** del try.

El programa preguntará el nombre de una provincia. Si el programa encuentra una línea que no tiene 5 campos, la imprimes así: " LÍNEA <nºLínea> INCORRECTA: <CONTENIDO_LINEA>". De las líneas que sean correctas (tengan 5 campos) y **contenga el nombre** de la provincia leído desde el teclado **(da igual en mayúsculas o minúsculas)** en el primer campo, imprimes el contenido y vas sumando las hectáreas. Cuando ya no haya más líneas, muestra un total de la cantidad de hectáreas de espacios naturales que tiene la provincia.

Si no encuentra la provincia, lo indica. Por último, cierra el fichero.

```
jrr@Prx206-01:/media/jrr/TOSHIBA EXT/2020-1DAM/U06-ficheros y flujos$ java U6E01.java

Espacios Naturales de la Provincia: alicante
Espacio: Cap de Sant Antoni Tipo: Reserva Natural Marina Superficie (Ha): 110,00
Espacio: Carrascar de la Font Roja Tipo: Parque Natural Superficie (Ha): 2500,00
Espacio: Fond, El Tipo: Parque Natural Superficie (Ha): 2495,00
Espacio: Illa de Tabarca Tipo: Reserva Natural Marina Superficie (Ha): 1440,00
Espacio: Llacunes de la Mata i Torrevieja Tipo: Parque Natural Superficie (Ha): 3708,00
Espacio: Marjal de Pego Oliva Tipo: Parque Natural Superficie (Ha): 1253,00
Espacio: Montg, El Tipo: Parque Natural Superficie (Ha): 2475,00
Espacio: Penyal dIfac Tipo: Parque Natural Superficie (Ha): 48,00
Espacio: Salines de Santa Pola Tipo: Parque Natural Superficie (Ha): 2574,00
Espacio: Serra de Mariola Tipo: Parque Natural Superficie (Ha): 17257,00
  LINEA 186 INCORRECTA: Ciudad Real;;;;
  LINEA 187 INCORRECTA: Ciudad Real;Crdoba;;;
  LINEA 380 INCORRECTA: LP (Gran Canaria);;;;
  LINEA 381 INCORRECTA: LP (Gran Canaria);LP (Lanzarote);;;
Total de superficie: 33860,00 Ha
```

EJERCICIO 2. FICHEROS Y EL USO DE BUFFERS.

APARTADO A (0.4 PUNTOS) Haz un programa llamado **U6E02A.java** que usando un JFileChooser permita escoger cualquier archivo para leerlo como un flujo de bytes. Usa un **FileInputStream()** para trabajar con él como un archivo de bytes y le calculas el histograma de frecuencias de cada byte. Es decir, cuentas el nº de veces que aparece cada byte en el fichero (recuerda que hay 256 bytes posibles que van del valor 0 al byte 255) leyendo byte a byte con la función **read()**. Cuando hayas leído todos los bytes del fichero y contado cuantas veces aparece cada uno y cuantos hay en total, divide cada valor entre el total de bytes leídos, eso te dará la probabilidad de encontrar cada byte en el fichero si escoges uno al azar.

El programa debe indicar :

- El peso del fichero con el que ha trabajado en bytes.
- El tiempo en segundos empleado en procesarlo (calcular su histograma).
- El propio histograma con valores de 5 dígitos de precisión y a 6 columnas.

```
jrr@Prx206-01:/media/jrr/TOSHIBA EXT/2020-1DAM/U06-ficheros y flujos$ java U6E02.java

Fichero: 1DAM-PROG-U06-Ficheros y Streams.pdf
Peso: 1028164 bytes. Proceso realizado en 1,160 segundos.
h[  0]: 0,009491 h[ 32]: 0,008559 h[ 64]: 0,003275 h[ 96]: 0,003421 h[128]: 0,003266
h[  1]: 0,003841 h[ 33]: 0,003748 h[ 65]: 0,003737 h[ 97]: 0,005039 h[129]: 0,003395
h[  2]: 0,003494 h[ 34]: 0,003660 h[ 66]: 0,004142 h[ 98]: 0,004551 h[130]: 0,003588
h[  3]: 0,003746 h[ 35]: 0,003651 h[ 67]: 0,003892 h[ 99]: 0,004421 h[131]: 0,003601
```

APARTADO B (0.4 PUNTOS) Copia el código del apartado anterior y crea el programa **U6E02B.java**, que básicamente hace lo mismo, pero ahora, usa un buffer de 64Kb (un array de bytes de 65536 elementos bytes) e intenta leer trozos del fichero de ese tamaño usando el método **read(buffer)** que devuelve la cantidad de bytes leídos y almacenados en el buffer. Vuelve a usarlo con el mismo fichero y responde a esta pregunta.

En ficheros grandes ¿Hay diferencia de tiempo al usar buffers en las operaciones de E/S? Muestra imagen de ejecución para el fichero .pdf de la unidad 6 ejecutado con ambos programas y en la misma máquina.

> **Nota**: ojo que los bytes en Java son valores con signo (-127 a +128) y los bytes que almacena el fichero de 0 a 255. Si trabajas directamente con un byte del buffer, si es negativo debes deshacer el complemento a 2: **256 + valor**.

APARTADO C (0.2 PUNTOS). Aprovecha el código más rápido y lo copias en el fichero **U6E02C.java,** elimina las sentencias que imprimen el histograma y haz que calcule la entropía de la información almacenada en el fichero.

La entropía H de un mensaje/información es la cantidad de bits de información útil que contiene realmente cada byte. Si obtienes un H de 8, significa que de media, cada bit de cada byte del mensaje contiene 1 bit de información útil. Si obtienes un valor inferior a 8, significa que hay bits que son redundantes, se pueden eliminar (si encuentras la forma de hacerlo, claro). Los compresores (como zip o rar) lo que hacen es intentar eliminar esta redundancia (equivale a subir la entropía H). La entropía de un mensaje X, se escribe H(X) y se calcula así:

$$H(x) = \sum_{i=1}^{n} p(i) \log \left(\frac{1}{p(i)} \right)$$

La entropía se mide en bits (bits útiles). Donde i es cada uno de los bytes del mensaje, p(i) es la frecuencia con la que el byte i aparece en el mensaje (nº de veces que aparece dividido por el total de bytes), y el logaritmo es en base 2 que se puede calcular así:

$$\log 2(x) = \frac{\log 10(x)}{\log 10(2)}$$

Si un byte i tiene p(i) a 0, no calcules la expresión, no lo sumes porque es 0. Por ejemplo en el mensaje X= "la casa es bonita" calcular H(X) consiste en contar cuantos bytes tiene (17), cuantas veces aparece cada byte (xi), luego calculas p(i) = xi / 17 ...

Byte	nºveces	frecuencia p(i)	log2(1/p(i))	p(i) * log2(1/p(i))
l	1	1/17 = 0.059	4.083	0.241
a	4	4/17 = 0.235	2.089	0.491
	3	3/17 = 0.177	2.498	0.442
c	1	1/17 = 0.059	4.083	0.241
s	2	2/17 = 0.118	3.083	0.364
e	1	1/17 = 0.059	4.083	0.241
b	1	1/17 = 0.059	4.083	0.241
o	1	1/17 = 0.059	4.083	0.241
n	1	1/17 = 0.059	4.083	0.241
i	1	1/17 = 0.059	4.083	0.241
t	1	1/17 = 0.059	4.083	0.241

La suma: 1.928 + 0.364 + 0.442 + 0.491 = 3.225 -> De cada 8 bits (si es ASCII y cada letra es un byte) 3.225 bits son información (entropía H) y 8 – 3.225 = 4.775 bits son redundantes -> un compresor podría bajar el peso de la información eliminando los bits redundantes.

Para el fichero que indiques (puede ser cualquiera) harás el mismo proceso. Mostrarás su peso en bytes, el tiempo que tarda en calcular su H. Cuanta entropía media tiene por byte (entre 0 y 8) cuantos bits son redundantes en cada byte (8 – entropía). Si la redundancia es:

- Menor de 0,1 bits => el mensaje es aleatorio o está comprimido.
- Si está entre 0,1 y 3,5, puede estar parcialmente comprimido
- De lo contrario, no está comprimido.

Ejemplo de ejecución:

```
Fichero: 1DAM-PROG-U06-Ficheros y Streams.pdf
Peso: 1028164 bytes. Proceso realizado en 0,03 segundos.
La entropía del fichero es: 7,979338 bits
La redundancia es 0,020662 bits
Fichero comprimido o aleatorio
```

EJERCICIO 3. TRABAJAR CON FICHEROS Y CARPETAS.

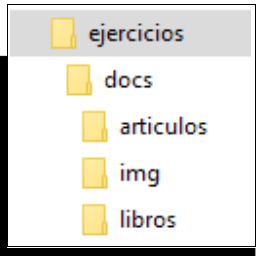
Haz el programa **U6E03.java** que imprima la ruta de la carpeta actual y pregunte por una carpeta y una extensión de archivo. Crea un filtro y recupera los ficheros que concuerdan con el filtro. Luego recupera de nuevo todos los ficheros que contine esa carpeta indicada y:

Para aquellos que sean directorios:

- Imprime su ruta absoluta.
- Vuelve a repetir la operación de manera recursiva indicando el número de ficheros que cumplen el filtro y la suma de su peso (en MiB).

Para aquellos que sean ficheros:

- Su nombre (sin ruta)
- Peso en Bytes.
- Fecha de última modificación. El método File.lastModified() devuelve un long, pero puedes usarlo para construir un Date a partir de él.
- R si es leíble y un guión si no lo es, W si es escribible y un guión si no lo es.
- El usuario propietario del archivo utilizando las clases java.nio.file.Path para generar un Path a partir de un io.File, la clase java.nio.file.Files y java.nio.file.attribute.UserPrincipal para pedirle el propietario mediante Files.getOwner(ruta).




```
E:\2020-1DAM\U06-ficheros y flujos>java U6E03.java
Carpeta actual: E:\2020-1DAM\U06-ficheros y flujos
Carpeta a examinar: ejercicios
Explora ficheros que acaban en: .pdf
[ejercicios]
  [docs]
    [articulos]
      GS.pdf 14828,995 KiB Ul.Mod. 29-01-2020 19:24:01 Permisos(RWX): R-X Propietario: pc/jose
    [img]
    [libros]
      [codigo]
      Shader.pdf 9907,768 KiB Ul.Mod 29-01-2020 19:05:20 Permisos(RWX): RWX Propietario: pc/jose
  Ejer.pdf 115,139 KiB Ul.Mod 23-03-2020 19:03:31 Permisos (RWX): RWX Propietario: pc/jose

Hay 3 ficheros con extension .pdf con un total de 24 MB
```

Observa que carpetas y ficheros dentro de una misma carpeta aparecen alineados

EJERCICIO 4. MAPAS EN UN FICHERO.

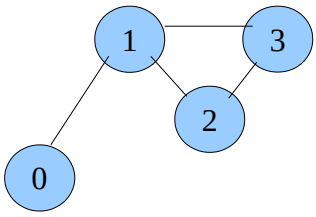
Crea el programa **U06E04.java** que pregunte por un fichero de texto con extensión **.GPS** y pueda crear un objeto Grafo a partir de los datos almacenados en el fichero indicado. Ejemplo de ejecución:

```
Símbolo del sistema

G:\2020-1DAM\U06-ficheros y flujos>java U6E04.java

Fichero (.GPS): carreteras.GPS
Origen: Badajoz
Destino: Valencia
  Badajoz (+403)-> Madrid (+251)-> Albacete (+191)-> Valencia Total: 845
Origen: Valencia
Destino:
Origen: Valencia
Destino: Bilbao
  Valencia (+191)-> Albacete (+251)-> Madrid (+395)-> Bilbao Total: 837
Origen: Bilbao
Destino: Bilbao
  Bilbao(0) -> Bilbao
Origen: Rubielos de Mora
Destino:
Origen: Rubielos de Mora
Destino: Madrid
  Origen Rubielos de Mora no encontrado
Origen:
Destino:

G:\2020-1DAM\U06-ficheros y flujos>
```

En el método main() definirá dos variables String origen y destino. Luego preguntará por el nombre de una ciudad de origen y otro nombre de ciudad de destino. Si ninguna está vacía, el programa intentará mostrar la ruta más corta para llegar al destino desde la ciudad de origen incluyendo la distancia de cada tramo y la distancia total a recorrer. Mientras las dos ciudades no estén vacías (**isEmpty()**), vuelve a preguntar por un origen y un destino y repite el proceso. Cuando ambos strings estén vacíos, el programa acaba.

LA CLASE GRAFO

Un grafo **G = (V,A)** está formado por un conjunto de **vértices** (**V**) y un conjunto de aristas (**A**). Los vértices se identifican mediante un número entero y pueden tener otra información adicional. Las aristas también pueden tener información adicional además de qué vértices son los que unen. Hay grafos dirigidos, si las aristas te obligan a seguir una sola dirección, o no dirigidos, si la existencia de una arista de v1 -> v2 implica que existe v2 -> v1. Por ejemplo:




```
Este grafo G= (V, A)
  V = { 0, 1, 2, 3 }
  A = { (0,1), (1,2), (1,3), (2,3) }

Es un grado no dirigido porque si (u,v) es una arista
de A, (v, u) también es una arista de A. Si las
aristas tienen un peso, se les llama ponderadas.
```

¿Cómo se representa un grafo en un programa?

Para los grafos que no tengan muchos vértices podemos usar una **tabla de adyacencias**. Consiste en hacer una tabla de tantas filas y columnas como número de vértices tenga el grafo y cuando en la celda [i][j] de la tabla de adyacencias haya un valor distinto de unValor, es que hay una arista que une a los vértices i y j. Su peso puede ser el valor que se guarda en la celda. Algún valor especial (unValor) representa que no hay arista entre ellos. Si el grafo es no dirigido, al crear la arista [i][j] también creas la arista [j][i] o bien solo marcas una de ellas pero asumes que la otra también está. Ejemplo: (en vez de true indicando que hay arista, podría haber un valor si las aristas son ponderadas). En el ejemplo, las celdas que no están a true, estarían a false (no las he puesto por simplificar).

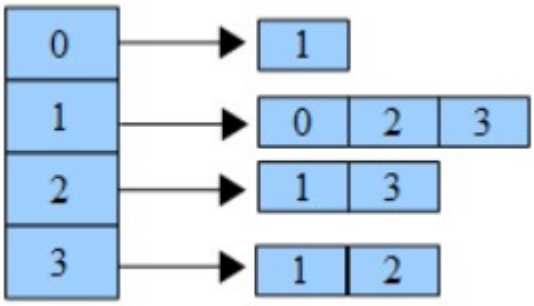
	0	1	2	3
0		true		
1	true		true	true
2		true		true
3		true	true	

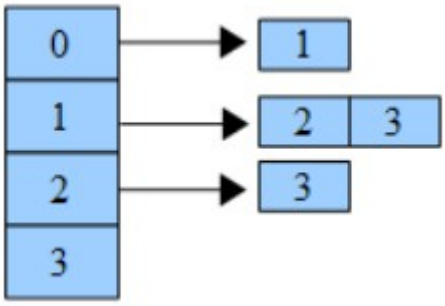
Tabla de adyacencias marcando aristas (i,j) y (j,i)

	0	1	2	3
0		true		
1			true	true
2				true
3				

Tabla de adyacencias marcando solamente (i,j) donde i < j

Si el número de vértices crece (los círculos), esta estructura es poco eficiente (desperdicia memoria) salvo que el grafo fuese muy denso (de media cada vértice se conecte a muchos otros) y sería preferible que cada vértice almacene su propia lista de vértices a los que se conecta. Incluso podría decidirse almacenar solo la pareja (u,v) donde u < v sabiendo que también existe la (v,u) o al contrario. Ejemplos de representación con listas de aristas (parecen arrays asimétricos, ¿verdad?):



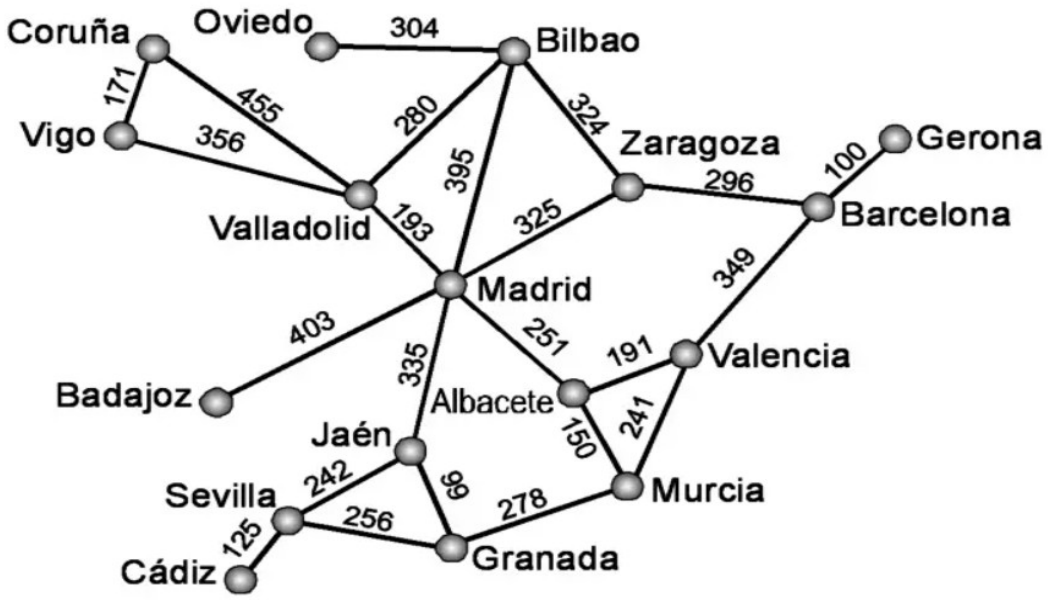


FORMATO DE NUESTROS FICHEROS .GPS

El fichero **carreteras.GPS** es un fichero de texto con la siguiente estructura:

```
nV                  --> entero con el Nº de vértices
nA                  --> entero con el Nº de aristas
0       Ciudad_0    --> índice del vértice y la ciudad que representa
           :                :
nV-1    Ciudad_vN-1 --> índice del vértice y la ciudad que representa
v1 v2 k_0           --> arista0 que une v1 y v2 a una distancia de k_0 km.
    :                           :
v1 v2 k_nA          --> arista nA-1 que une v1 y v2 a una distancia de k_nA km
```

Y representa el siguiente mapa de carreteras:



Haz una clase llamada Grafo que no sea pública y la puedes hacer en el mismo fichero que la clase ejecutable y que tenga estas características:

Estado:

- Los enteros **nV** y **nA** (nº de vértices y nº de aristas del grafo respectivamente).
- Un array **V** de **String** para guardar el nombre de cada vértice (la ciudad).
- Una tabla de enteros llamada **tA (tabla de adyacencias)** para las aristas.

Contructores:

- Un constructor que acepte un string como nombre de un fichero de texto de tipo .GPS y lea el fichero y cree el objeto a partir de él. Cuando leas el fichero, hazlo de forma que se proteja de posibles errores en los ficheros que lee (nº de vértice por encima o igual que nV), lo mismo con las aristas (que indiquen vértices que no existen), que esté incompleto y se acaba antes de definir un vértice o una arista... (en esos casos, muestra por consola el error "ERROR FORMATO en <Fichero> LÍNEA <nºLínea>".

  ***Nota**: Cuando leas Strings de ficheros de texto, aplica un trim() para eliminar posibles espacios en blanco por delante y detrás o puede que tengas problemas para localizar los vértices.*

Métodos:

- **String caminoMasCorto(String origen, String destino):** método público a nivel de objeto al que le pases el nombre de dos ciudades y devuelva el camino más corto y cuántos kilómetros tiene, que sale de origen y acaba en destino. Su comportamiento será:

- Si no existe la ciudad origen o destino como vértice, devuelve "ciudad origen/destino <tal> no encontrada".
- Si la ciudad origen es igual que la ciudad de destino devuelve "<origen> (+0)-> <destino>".
- Si origen y destino son diferentes y puedes localizar su número de vértice llama al método **cmcDijkstra(int o, int d)** (camino más corto usando el algoritmo de Dijkstra) y devuelve el string que retorna esta función.

- **String cmcDijkstra(int o, int d):** Este método será de uso interno (oculto al exterior) y lo utiliza el anterior. Recibe de argumentos el vértice de origen y de destino y utilizando el algoritmo de Dijkstra buscará el camino más corto para llegar desde o a d (corto en el sentido de que la suma de pesos sea mínimo) con un orden (esfuerzo) de O(n^2) donde n es el nº de vértices. El algoritmo es este:

CONSEJO: Hazte también la clase ConjuntoEnteros con estos componentes:

Datos:
n: entero con el nº de elementos que tiene actualmente ocupando las posiciones de 0 a n-1.
c: array de enteros que almacena los elementos desde la posición 0 hasta la posición n-1.
Constructor:
ConjuntoEnteros(int max): max es el mayor nº de elementos que puede tener.
Métodos:
inserta(int e) Si e ya está, no hace nada. Si no, si no cabe, lanza una excepxión "ERROR: capacidad superada al añadir <e>". En otro caso, guarda el elemento e en la posición c[n] e incrementa n en 1.
booleano pertenece(entero e): devuelve true si el elemento e está en el conjunto y false en caso contrario.
elimina(int e) Si el elemento e no está en el conjunto, no hace nada. En caso contrario, desplaza los elementos que haya a la derecha una posición a la izquierda y decrementa en 1 la variable n.

```
ConjuntoEnteros c1 = new ConjuntoEnteros(5);
c1.inserta(7); c1.inserta(4); c1.inserta(5); c1.inserta(1); c1.elimina(4);
```

entero getElemento(int i) devuelve el i-ésimo elemento, si i es menor de n.
boleano esVacio() devuelve true si no tiene elementos y false si tiene alguno.
Entero cardinal() es el getter de n, devuelve el nº de elementos que tiene actualmente.

ALGORITMO DE DIJKSTRA PARA EL GRAFO G = (V, A)

```
// ENTRADAS:
//    o (nº de vértice de V, origen)
//    d (nº de vértice de V, destino)
// SALIDA: cadena indicando ruta a seguir saliendo de o y llegando a d
DEFINE S     // un conjunto de vértices de máximo nV elementos.
DEFINE D     // un array de nV enteros que almacena el coste para llegar a cada
             // vértice del grafo desde el vértice o
DEFINE P     // Otro array de nV enteros donde cada elemento i indica cuál es su
             // predecesor en el mejor camino
```

```
S <- S + { o }                  // Inicialmente S contiene el vértice origen
PARA i = cada vérticesdel grafo HAZ
      SI i no es el origen y existe la arista (o -> i) ENTONCES
            D[i] <- tA[o][i]          // distancia de origen al vértice i
      SINO
            D[i] <- MAYOR_ENTERO      // Mayor entero posible, no hay ruta
      FINSI
FINPARA
// Explorar el grafo...
PARA i = cada vértice del grafo HAZ
      SI S tiene todos los vértices del grafo ENTONCES sal del bucle FINSI
      w <- el vértice de V – S (w no está en S) que tiene el menor D[w]
      S <- S + {w}              // Se añade w al conjunto S
      PARA v = cada vértice de V – S que tiene arista (w -> v) HAZ
            SI D[v] + tA[w][v] < D[v]  ENTONCES
                  D[v] <- D[w] + tA[w][v]   // Es más corto llegar a v por w
                  P[v] <- w                 // A v se llega a través de w
            FINSI
      FINPARA
FINPARA
// Genera la ruta recorriendo P al revés, desde P[d] (el destino) al origen
distancia <- 0
camino <- V[d]            // La última ciudad
v <- d
MIENTRAS v != o HACER
      tramo <- tA[v][ P[v] ]
      distancia <- distancia + tramo
      camino <- v[ P[v] ] + " (tramo)-> " + camino
      v <- P[v]
FINMIENTRAS
camino <- camino + " Total: " + distancia
DEVOLVER camino
FIN ALGORITMO
```

EJERCICIO 5. HACER UN COMPRESOR DE FICHEROS CONCRETOS.

Comprimir una información consiste en cambiar la forma de codificarla para **aumentar su entropía**, es decir, usar menos bits (pero mejor aprovechados) para que contengan la misma información y esto se traduce en bajar el peso. Vamos a hacer un compresor y descompresor sencillo orientado a un tipo concreto de archivos de texto aprovechando que sabremos de antemano lo que contiene. El programa transforma ficheros de texto con pocos símbolos diferentes en ficheros binarios codificados y también puede hacer la operación contraria para recuperar la información original.

El ADN está formado por 4 moléculas llamados nucleótidos (Adenina, Guanina, Tanina y Citosina), que se suelen representar por las letras AGTC respectivamente. Cuando en un fichero se almacenan estas cadenas de letras en ASCII estamos empleando 8 bits para cada una. Pero como solo hay 4 letras, podríamos codificar cada letra en 2 bits (00=A, 01= G, 10 =T

y 11=C) aumentando la entropía (empaquetando 4 letras en cada byte) y **usando un último byte de control en el fichero comprimido** para controlar estos casos:

```
...ATCA      -> 00101100 + 00000100 (último byte 0 --> anterior tiene 4 letras)
...ATC       -> 00101100 + 00000001 (último byte 1 --> anterior tiene 1 letra)
...AT        -> 00100000 + 00000010 (2 indica que hay 2 letras)
...A         -> 00000000 + 00000011 (3 indica que hay 3 letras)
```

¿Cómo empaquetar 4 letras ASCII (AGTC) en un solo byte? Aunque solo guardemos trozos de 8 bits del entero (los menos significativos), utiliza un tipo int para no tener problemas con el signo de los bytes de Java.

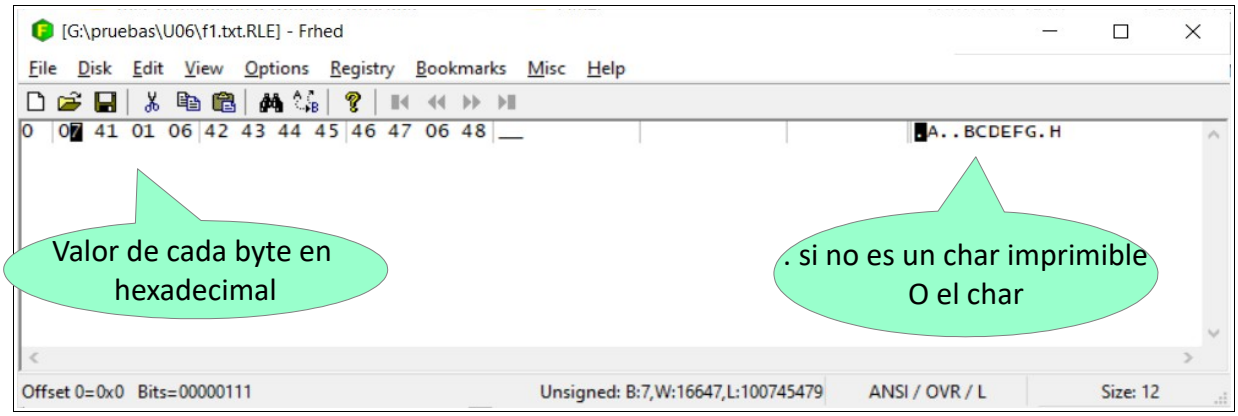
```
Primera letra ---> 00 / 01 / 10 / 11 = AA                              ---> 000000AA
Desplaza bits 2 posiciones a la izquierda / multiplica por por 4       ---> 0000AA00
Si hay segunda letra ---> 00 / 01 / 10 / 11 = BB la sumas              ---> 0000AABB
Desplaza bits 2 posiciones a la izquierda / multiplica por por 4       ---> 00AABB00
Si hay tercera letra ---> 00 / 01 / 10 / 11 = CC la sumas              ---> 00AABBCC
Desplaza bits 2 posiciones a la izquierda / multiplica por por 4       ---> AABBCC00
Si hay tercera letra ---> 00 / 01 / 10 / 11 = DD la sumas              ---> AABBCCDD
Crea el Byte de control: 4 si hay 4 letras,  3 si hay 3, 2 si hay 2 y 1 si hay 1.
```

El desempaquetado de cada byte es similar, mientras haya bytes coges uno, extraes cada 2 bits y regeneras la letra: anulas el resto de bits (AND $00000011_{2)}$) tienes letra4, desplazas 2/divides por 2 y anulas el resto (AND $00000011_{2)}$) obtienes letra3, desplazas y anulas y tienes letra2…

Haz el programa **U06EP05ADN.java** que acepte/lea un fichero de texto con extensión .ADN y pueda comprimirlo y descomprimirlo:

java U06EP05ADN c fichero.ADN --> comprime en fichero.ADC
java U06EP05ADN d fichero.ADC --> descomprime en fichero.ADN

Debes intentar que el programa sea rápido porque estos ficheros tienden a ser grandes. Además no estaría mal que mientras lo desarrollas, te crees ficheros de texto pequeños de tipo .ADN para probar que procesa bien todos los casos. Utiliza un editor hexadecimal, son programas útiles cuando tienes que trabajar con ficheros binarios, yo te recomiendo http://frhed.sourceforge.net/ free y sencillo.



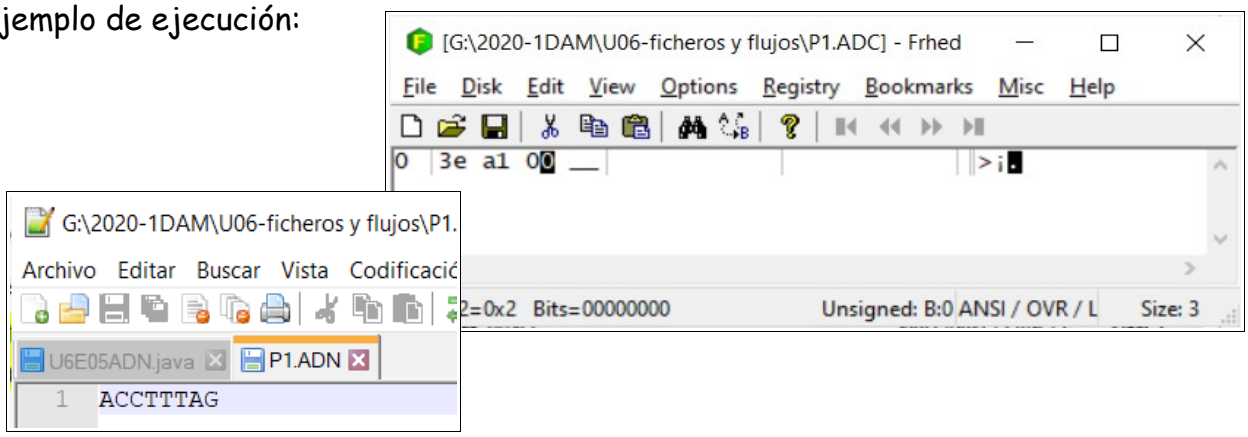
Cuando te funcione bien, rellena la siguiente tabla tras comprimir los ficheros **ecoli.ADN** (una bacteria) y **c14H.ADN** (el cromosoma 14 de los humanos) con tu programa y con un compresor

que tengas en tu sistema en formato .zip. De cada fichero (.ADN, .ADC y .ZIP) calcula su H (entropía por byte) y peso (en MegaBytes).

Fichero	Peso	H	Fichero:	Peso	H	Fichero:	Peso	H
ecoli.ADN			ecoli.ADC			ecoli.zip		
c14H.ADN			C14H.ADC			C14H.zip		

Me pasas el código, la tabla rellena, capturas de pantalla de comprimir estos ficheros y una conclusión sobre el resultado que consigue tu programa en comparación con el compresor que hayas usado en tu equipo.

Ejemplo de ejecución:



EJERCICIOS JEDI (EL QUE TE TOQUE)

Como siempre, aunque solo tenéis que entregar el que os toca, debes intentar hacerlos todos.

EJERCICIO 6. TEXTO CON EXPRESIONES REGULARES.

Si usamos la clase **Scanner** para la lectura de datos de un fichero de texto, aprovechamos que muchos de sus métodos están diseñados para tratar la entrada de datos dividiéndola en bloques separados por caracteres espaciadores (delimitadores).

El delimitador por defecto es cualquier carácter de tipo espacio (espacios en blanco, tabuladores, saltos de línea). En según qué casos, nos puede interesar utilizar otro tipo de separador, valores separados por coma (Comma Separated Values, CSV) y un salto de línea ("\n") indica el fin de una fila de datos. O incluso podemos proporcionar expresiones regulares para indicar qué queremos usar como delimitadores.

Para modificar el "delimitador" al punto y coma ";" creamos un patrón (objeto **Pattern** que describe como es el separador) y le indicamos a la clase Scanner que use este patrón como delimitador mediante el método '**useDelimiter()**':

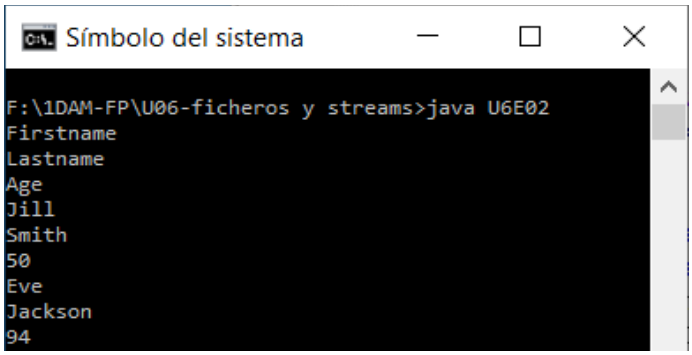
```
Pattern delimitador = Pattern.compile(";");
scaner.useDelimiter( delimitador );
```

Un vez cambiado el delimitador, leemos el fichero utilizando un bucle, comprobando si quedan elementos por leer con **hasNext()** y leyendo cada cadena con **next()**. Sin embargo tendremos problemas con los saltos de línea. Por tanto habrá que indicar que el salto de línea también es un separador:

```
Pattern delimitador = Pattern.compile("[;\n]");  // Ahora pasamos una expresión regular
```

Las expresiones regulares las comentamos en la unidad 4. Te aconsejo que les eches una ojeada de nuevo. Debes hacer un programa que lea un fichero html (se supone por simplificar, que no tiene CSS ni javascript) e imprima por pantalla el texto que contiene eliminando todas las etiquetas html. Para ello, piensa como definir una expresión regular que represente cualquier etiqueta html y la usas para crear un patrón. Ese patrón se lo das a un scanner para que haga de delimitador y procesas el fichero imprimiendo por pantalla todo lo que no sea una marca HTML, pero antes eliminas posibles espacios (método **trim()** de un String). Por ejemplo si lo aplicas a este fichero html, el resultado será:

```
<html> <body>
<table style="width:100%">
  <tr> <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>    <td>Jill</td>
    <td>Smith</td>
    <td>50</td>
  </tr>
  <tr>    <td>Eve</td>
    <td>Jackson</td>
    <td>94</td>
  </tr>
</table> </body> </html>
```



EJERCICIO 7. FICHERO BINARIO DESDE TEXTO BASE64.

Muchos protocolos de red (como los de mail) solo mueven texto y cuando necesitan mover ficheros binarios deben convertirlos a texto e incrustar este texto en el mensaje y en el destino, regenerar el contenido binario a partir de este texto. Haz el programa **U6E07.java** que pregunte por un fichero a convertir, leerlo como un fichero binario. Luego pregunta por el nombre de un fichero de texto a generar donde incrusta este contenido y que tendrá este formato:

FICHERO=nombrefichero.extensión BYTES= .... letras con sus datos en base64 ... FINBYTES

Para convertir un fichero binario a base64 debemos comprender que este sistema de numeración utiliza 64 letras para escribir los valores de 0 a 63. Es decir, 6 bits de información. Aquí tienes la tabla de equivalencia:

Value	Char	Value	Char	Value	Char	Value	Char
0	A	16	Q	32	g	48	w
1	B	17	R	33	h	49	x
2	C	18	S	34	i	50	y
3	D	19	T	35	j	51	z
4	E	20	U	36	k	52	0
5	F	21	V	37	l	53	1
6	G	22	W	38	m	54	2
7	H	23	X	39	n	55	3
8	I	24	Y	40	o	56	4
9	J	25	Z	41	p	57	5
10	K	26	a	42	q	58	6
11	L	27	b	43	r	59	7
12	M	28	c	44	s	60	8
13	N	29	d	45	t	61	9
14	O	30	e	46	u	62	+
15	P	31	f	47	v	63	/

Como en un fichero binario tenemos bytes, debemos juntar 3 bytes (24 bits) para generar 4 letras en base64. Según la cantidad de bytes que haya en el fichero podemos encontrarnos estas 3 posibilidades al acabar de procesar el fichero:

La primera posibilidad (la cantidad de bytes es múltiplo de 3):

- Los primeros 6 bits del primer byte generan la primera cifra base64 (A).
- Los 2 bits menos significativos del primer byte pegados a los 4 más significativos del segundo byte generan la segunda cifra base64 (B)
- Los 4 bits menos significativos del segundo byte y los dos más significativos del tercer byte generan la tercera cifra base64 (C)
- Los 6 bits menos significativos del tercer byte forman la cuarta cifra base64 (C).

Otro caso ocurre cuando a la cantidad de bytes en el fichero le falta un byte para ser múltiplo de 3:

- Los primeros 6 bits del primer byte generan la primera cifra base64 (A).
- Los 2 bits menos significativos del primer byte y los 4 más significativos del segundo byte generan la segunda cifra base64 (B)
- Los 4 bits menos significativos del segundo byte y un relleno de 2 ceros generan la tercera cifra base64 (C)

- Se indica que falta 1 byte con un carácter '=' para que se sepa que se ha usado relleno.

El último caso ocurre cuando a la cantidad de bytes en el fichero le faltan dos bytes para ser múltiplo de 3:

... | 6 + 2 bits | -> ... AB==

- Los primeros 6 bits del primer byte generan la primera cifra base64 (A).
- Los 2 bits menos significativos del primer byte y un relleno de 4 ceros a la derecha generan la segunda cifra base64 (B)
- Se indica que faltan 2 bytes con dos caracteres '==' para que se sepa que se ha usado relleno de 4 bits a 0.

Ejemplo:

... | 10101010 | 11001100 | -> ... 101010 - 101100 - 110000 -> 42 - 44 - 48 -> qsw=

Ejemplo de ejecución:

```
Símbolo del sistema
F:\1DAM-FP\U06-ficheros y streams>java U6E07.java

Teclee el Fichero binario: bender.png

Teclee el Fichero de texto de salida: bender.txt
```

Resultado (ojo no comprobad con notepad, mejor otro editor). Podéis comprobar los códigos iniciales y finales que genera vuestro programa para verificar si funciona de forma correcta:

FICHERO bender.png **BYTES**=iVBRw0Ggo ... IAAAAElTkSQmC **FINBYTES**

EJERCICIO 8. SERIALIZAR Y DESERIALIZAR.

Crea el programa **U6E08.java**, que presenta al usuario este menú y que modifique la clase Grafo para que sea Serializable:

```
GRAFOS
=====

1. Leer.
2. Guardar.
3. Modificar.
4. Camino Corto.
5. Salir
```

```
LEER GRAFO
==========

1. De fichero de texto.
2. Deserializar .OBJ
3. Volver al principal
```

```
Elija opcion (1-3): 1
Fichero .GPS:
```

```
Elija opcion (1-3): 2
Fichero .OBJ:
```

```
GUARDAR GRAFO
=============

1. A fichero texto.
2. Serializar.
3. Volver al principal
```

```
Elija opcion (1-3): 1
Fichero .GPS:
```

```
Elija opcion (1-3): 1
Fichero .OBJ:
```

```
    CONSULTA
    ========

Origen: Oviedo
Destino: Valencia
```

```
    MODIFICA GRAFO
    ==============

1. Añadir Arista.
2. Borrar Arista.
3. Modificar Peso Arista.
3. Volver al Principal
```

Antes de poder utilizar las opcines 2, 3 y 4, debes haber leido un grafo desde la opción 1. Debes controlarlo con una variable booleana y no hacer aparecer las opciones hasta que se puedan utilizar. La opción 1 del submenú 1 ya la tienes hecha en el ejercicio anterior.

La opción 2, subopción 1, te permite guardar el grafo en un fichero de texto .GPS. Consulta el formato en el ejercicio anterior para conseguirlo.

La opción 2, subopción 2 te debe permitir exportar el objeto (tienes que haberlo convertido en serializable) de forma completa a un fichero con extensión .OBJ.

La opción 3, subopción 1 te permite añadir una arista al grafo. Pregunta por el índice del primer vértice (v1), pregunta por el índice del segundo vértice (v2). Si los vértices no existen, o ya existe l arista (v1, v2) indicas el error y vuelves al menú. SI no hay problemas, pregunta por el peso de la arista y la creas.

La subopción 2 es similar, pero para borrar una arista y para modificar su peso. Valida las operaciones para evitar posibles errores y las realizas.

Para probarlo:

- Lee el fichero carreteras.GPS
- Modifica el grafo y añade la arista entre Valencia y Zaragoza de 309
- Lo guardas como carreteras1.GPS en modo texto y como carreteras1.OBJ serializándolo.
- Cierra el programa y lo vuelves a cargar primero desde el fichero de texto y luego desde el fichero objeto.
- Por último comprueba la mejor ruta para ir de Valencia a Zaragoza.

EJERCICIO 9. FICHEROS DE DATOS DE ACCESO ALEATORIO.

APARTADO a) (0.5 PUNTOS) Un fichero de índice (índice secundario) es un fichero que ayuda a recorrer los registros de otro fichero (el fichero de datos, fichero maestro o fichero principal) en cierto orden (el de la clave) sin necesidad de reordenar los registros en el fichero maestro. Esto ahorra espacio (no tienes que tener los mismos registros almacenados en otro orden) y tiempo (ordenar ficheros largos consume mucho tiempo). Un índice es una serie de parejas:

(clave, posición del registro en el fichero maestro)

donde la clave es uno o varios campos por los que implementas el orden. Nosotros vamos a suponer que solo indexamos por un campo.

Mira el fichero **espacios3.csv** y observa que todas las líneas (registros) y campos tienen la misma longitud (tamaño fijo). Usa este tamaño para ir recorriendo el fichero registro a registro.
Haz el programa **U609A.java** que lea este fichero de texto y cargue algunos datos en un array de objetos de tipo **EntradaIndiceString** cuya clase tendrás que definir de esta manera:

```
EntradaIndiceString:
   String clave        // Sirve para ordenar por este valor los registros
   int  numRegistro    // Permite calcular el byte del fichero donde comienza el registro
```

De clave usarás el String que hay en la provincia y en numRegistro guardas el nº de registro dentro del fichero (0, 1, 2, ...). Para saber cuantos registros hay (y por tanto el tamaño del array) puedes dividir el peso del fichero entre la longitud de cada registro.
Cuando tengas el array de estos elementos, los ordenas de menor a mayor por la clave y generas un nuevo fichero binario llamado "**provincias.idx**" donde cada línea (registro) también tendrá longitud fija y estará formada por el nombre de la ciudad (un String) y el nº de registro en el fichero (un int).

c) (0.5 PUNTOS) Ahora crea el programa **U6E09C.java** que muestre este menú e implemente sus operaciones de forma que use el fichero de índice del apartado A cuando se active (opción 5) o el orden del fichero de datos cuando se desactive:

```
[1] Primero [2]Anterior [3]Siguiente [4]Ultimo  [5]Activa/desactiva índice [6] Salir

Registro actual: 7
Índice Activado: false
Datos:
+-------------------------+-------------------------------+-------------+-------------+--------+
| PROVINCIA               | ESPACIO                       | CCAA        | TIPO        |SUP(Ha) |
+-------------------------+-------------------------------+-------------+-------------+--------+
|A Coruña                 |Fragas do Eume                 |Galicia      |Parque Natura|    9073|
+-------------------------+-------------------------------+-------------+-------------+--------+
```

EJERCICIO 10. COMPRESOR RLE.

Vamos a intentar programar un compresor **genérico** (que sirva para cualquier tipo de fichero) y **sin pérdidas** (comprime y vuelve a descomprimir recuperando la misma información que comprimió), al contrario que el del ejercicio 5 (solo sirve para ficheros .ADN), sirve para cualquier tipo de fichero.
Usará la codificación **RLE (Run Length Encoding)** que permite comprimir información cuando la redundandcia es alta y hay largas secuencias de bits/bytes repetidos. Por ejemplo considera la siguiente secuencia de bytes:

AAAAAABCDEFGGGGGHGGGG --> 21 bytes de peso

Tienes una primera secuencia de 6 A, luego 5 bytes que no forman secuencias, luego otra secuencia de 5 G, un solo carácter H y una última secuencia de 4 G. A cada secuencia la llamamos run, en el ejemplo tenemos runs de longitud 6, 1, 5 y 4. Si usamos 1 byte para guardar la longitud de una secuencia (encerramos entre corchetes para indicar que es un valor numérico en un byte) podemos representarla con el formato [longitud]byte:

[6]A[1]B[1]C[1]D[1]E[1]F[5]G[1]H[4]G --> 18 bytes de peso

Algunas mejoras y aspectos que hay que fijar para que esto pueda funcionar bien:

- **¿Podemos representar runs de longitud arbitraria?** No. Los run máximos que podemos representar serán de 255 bytes repetidos, si hay uno mayor habrá que romperlo en varios. Ejemplo: Un run de 520 letras Z como "ZZZZ...ZZZ" hay representarlo así: [255]Z[255]Z[10]Z (el motivo es que el mayor valor que podemos guardar en un byte es 255 y para guardar mayores valores debemos usar 2 bytes, lo que introduce más sobrecarga para runs pequeños).
- **¿Qué hacer con trozos donde cada byte es diferente del anterior?** No merece la pena codificarlos porque introducimos sobrepeso. Ejemplos: AB --> [1]A[1]B (acabas con 4 bytes cuando tenías 2); AAABABABA ---> [3]A[1]B[1]A[1]B[1]A[1]B[1]A --> acabas con 14 bytes y tenías 9. Por tanto vamos a adoptar una estrategia diferente para estos casos. Vamos a usar el byte [0] como un indicador de que a continuación hay cierta cantidad de bytes diferentes que no se van a codificar: [0][n]B1..Bn que significa: aparecen n bytes sin codificar que son B1..Bn. Usamos [0] porque nunca vamos a anotar un run de longitud 0, no existe el run. Así los ejemplos anteriores quedarían:

```
original     inicialmente                          mejora_propuesta
AB           [1]A[1]B (4 bytes)                    [0][2]AB (4 bytes)
AABABABA     [2]A[1]B[1]A[1]B[1]A[1]B[1]A (14 bytes)[0][8]AABABABA (10 bytes)
```

- **¿Cuál será la longitud mínima de un run?** Para considerar que varios bytes forman un run deben ser al menos 3 bytes repetidos, si son 2 no los consideramos run y por tanto se consideran como bytes distintos.

Implementa dos métodos en la clase de la aplicación: ambos deberían usar buffers para acelerar el proceso y trabajar con ficheros binarios.

comprime(): lee un archivo de cualquier extensión y lo procesa como binario, generando otro archivo con el mismo nombre y extensión que el original pero añadiéndole al nombre .RLE

expande(): abre un archivo .RLE y lo procesa en formato binario y genera otro cuyo nombre tiene el prefijo "D-" y se le quita el sufijo ".RLE" y contiene el resultado de recuperar los bytes originales a partir de los runs codificados.

Crea un método main() que permita aceptar parámetros:

```
java comdecomRLE.java c fichero.XXX      --->  comprime en fichero.XXX.RLE
java comdecomRLE.java d fichero.XXX.RLE  --->  descomprime en D-fichero.XXX
```

Muestra el nombre y el peso en Megas con 3 decimales de precisión del fichero original. Indica además cuanto tiempo tardas en comprimirlo en segundos con 3 dígitos de precisión. Indica como se llama el fichero comprimido y cuanto pesa en Megas y 3 dígitos de precisión. Calcula por último en qué % baja de peso (a veces puede tener sobrepeso):

```
G:\java ComdecomRLE.java c i1.bmp
  Fichero i1.bmp de 0,050 MB comprimido en 0,096 seg
  Destino i1.bmp.RLE de 0,020MB
  Baja de Peso un 39,655%
```

Comprime los ficheros:

- i1.bmp e i1.jpg ¿En cuál comprime más?

Descomprime el fichero i2.bmp.RLE

- ¿Qué imagen se ve?